



Adaptive key partitioning in distributed stream processing

Gang Liu¹ · Zeting Wang¹ · Amelie Chi Zhou² · Rui Mao¹

Received: 11 June 2023 / Accepted: 4 December 2023 / Published online: 12 January 2024
© The Author(s) 2024

Abstract

In stream processing systems, Key Grouping is a commonly employed partitioning scheme for distributing input tuples among parallel instances of stateful operators. With key grouping, tuples shared public keys in the stream are designated to the specific instance responsible for that key. Typically, the implementation of key grouping involves the use of a hash function. While it is convenient and deterministic, it is also known to cause load imbalance between parallel instances, especially in the presence of skewed data streams. Key-Splitting is an effective technique that distributes tasks associated with keys to downstream operators, facilitating load balancing at a relatively low cost. However, overly increasing parallel instances can lead to excessive aggregation costs, becoming a system bottleneck. In this paper, we show the high aggregation cost brought by the Key-Splitting partitioner at different levels of key separation. To address this challenge, we introduce an adaptive Key-Splitting method which controlling the degree of key separation. We propose a partitioner named FlexD, which aims to achieve dynamic adaptation of key separation limits for streaming data. The partitioner employs key grouping to distribute rare keys and dynamic expansion of processing instances to distribute hot keys. We implemented our method on Apache Storm and evaluated it by using real-world and synthetic datasets. Experimental results show that our method achieves a good balance between load balancing and aggregation cost. Moreover, it outperforms existing methods, achieving higher throughput.

Keywords Stream processing · Stream partitioning · Load balancing · Key grouping

1 Introduction

A streaming application is typically represented as a directed acyclic graph (DAG) in the streaming programming model. In this model, operators are represented as vertices, and data streams are represented as edges. Although task and pipeline parallelism are naturally exposed in this model, the

utilization of data parallelism is often limited. To address this limitation, researchers have proposed the concept of auto-parallelization (Schneider et al. 2012; Gedik et al. 2014) as a means to automatically and efficiently extract data parallelism in streaming applications. Specifically, auto-parallelization, also termed as auto-fission in Gedik (2014), aims to transparently identify and exploit data parallelism.

The stream processing system employs auto-parallelization to convert the logical graph of the streaming application into a parallelized execution graph before the deployment phase. Each operator in the parallelized graph is instantiated with multiple parallel instances, and a partitioning strategy for specific data is applied to assign each instance with a sub-stream of tuples from the upstream operator. Subsequently, the results from these parallel instances are typically merged back into a single stream while maintaining the original order.

The distribution of tuples from a single stream among the parallel instances of a target operator is determined by the operator's processing logic (Nasir et al. 2015a). There are two types of operators: *stateless* and *stateful*. For stateless

✉ Amelie Chi Zhou
amelieczhou@hkbu.edu.hk

Gang Liu
gliu@szu.edu.cn

Zeting Wang
2110276138@email.szu.edu.cn

Rui Mao
mao@szu.edu.cn

¹ Guangdong Province Key Laboratory of Popular High Performance Computers, College of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China

² Hong Kong Baptist University, Kowloon Tong, Hong Kong

operators, shuffle grouping is the preferred partitioning scheme. A commonly implemented shuffle grouping is the round-robin policy, which ensures load distribution balance among the parallel instances. On the other hand, stateful operators associate their state with one or more keys extracted from the tuples. When parallelizing stateful operators, the operator's state needs to be partitioned and maintained across multiple instances. Each instance focuses on the subset of tuples that affect its corresponding state partition. Key grouping is an effective partitioning scheme for stateful operators (Chen et al. 2021), as it assigns tuples with the same key to the same instance. This greatly simplifies the development of parallel stateful operators.

Key grouping struggles with skewed data streams (Nasir et al. 2015b; Pacaci and Özsu 2018). When data distribution is skewed, some downstream operators may be overwhelmed with data, while others might process too little. This imbalance in the system has a negative impact on its overall performance and utilization of computing resources. The skewed data stream poses various challenges. Key grouping techniques rely on uniformly distributed data to evenly distribute data among downstream operators. In skewed data streams, certain key values may occur more frequently than others, resulting in hot spot key values being sent to the same downstream operator, while other operators receive less data. The overloaded operators must process a larger amount of data, resulting in processing delays and potential performance degradation. In addition, the excessive load on some operators may cause underutilization of computing resources, as other operators are not fully utilized, resulting in wasted computing power and idle storage space. This imbalanced distribution of data and workload prevents the system from fully utilizing all available computing resources, resulting in overloaded nodes and idle nodes coexisting in the system.

In scenarios where there is a change in the characteristics of the input stream, certain solutions (Gedik 2014; Fang et al. 2017) adopt a solution that involves assessing the load imbalance and subsequently deciding whether to perform state migration to alleviate the imbalance while preserving the semantics of key grouping. These solutions do not rely on any prior knowledge of the stream composition. On the contrary, they dynamically adapt to changes in the input stream. In the case of frequent changes in the input stream, the adaptation process, including constructing new mapping functions and migrating states, can significantly affect the performance of the system. In addition, reactive partitioning incurs periodic overhead due to the continuous detection of load imbalances, even if the distribution of the input stream remains stable over time.

In recent years, numerous studies (Xiao et al. 2023; Abdelhamid and Aref 2020; Abdelhamid et al. 2020; Toliopoulos and Gounaris 2020; Zhang et al. 2017, 2019) on stream

processing have emerged, and the Key-Splitting method has become a prevalent solution in stream partitioning. However, these methods exhibit certain deficiencies, namely, an inability to adjust to changes in data distribution or achieve satisfactory load balancing, while scattering key values redundantly over numerous downstream operators, resulting in excessive aggregation overhead. Pacaci and Özsu (2018) and Katsipoulakis et al. (2017) have recognized the aggregation cost that arises from tasks being dispersed to different downstream operators. However, they have not been able to effectively limit heavy hitters from being continuously distributed to more downstream operators. We conducted a study on the proportion of the average aggregation time of heavy hitters to the average processing time when they are split to downstream operators at different levels. As shown in Fig. 1, the aggregation cost increases with the level of splitting. It is evident that splitting too many hot-keys to downstream operators to alleviate load balancing will lead to excessive aggregation cost, which affects the overall system performance. This conclusion is consistent with that of (Qureshi et al. 2019), which indicates that increasing parallelism does not necessarily guarantee better performance for tasks with state when aggregation is involved.

To address these issues, we introduce FlexD, a lightweight stream partitioning method. Similarly to previous work (Gedik 2014; Fang et al. 2017; Pacaci and Özsu 2018), our method adopts a hybrid strategy. FlexD utilizes Hash grouping to partition low-frequency keys and uses a progressive Splitting method to expand partitioning dynamically based on changes in key value frequency for high-frequency keys. The contribution of our work can be summarized as follows: First, we propose FlexD, which can dynamically adjust changes in data stream distribution to achieve significant load balancing metrics and improve the overall throughput of the system. Second, we verify the effectiveness of FlexD through extensive experimentation performed on synthetic and real datasets. The experimental results illustrate that FlexD significantly outperforms existing methods for various data distribution scenarios.

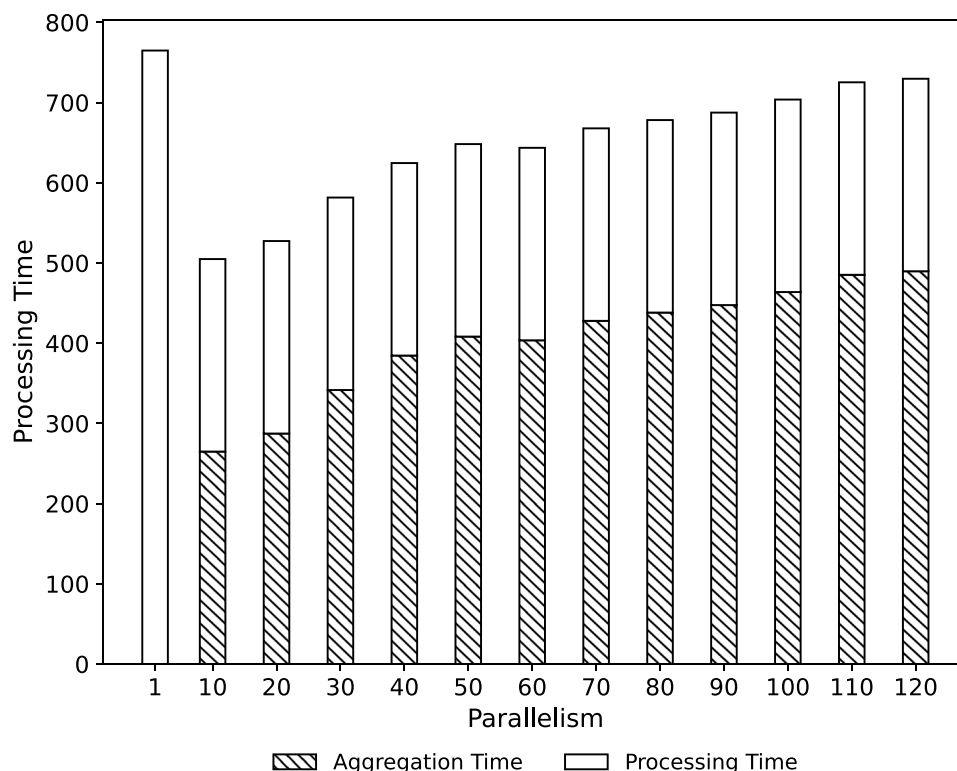
The rest of the paper is organized as follows. In Sect. 2, we review the related work. In Sect. 3, we describe the system model and the research problem. In Sect. 4, we present the design architecture of FlexD and the FlexD algorithm. We analyze the spatio-temporal complexity of the FlexD algorithm (Sect. 5) and extensively evaluate the adaptive capability and performance of FlexD on synthetic and real datasets in Sect. 6. Finally, we provide a summary in Sect. 7.

2 Related work

2.1 Shuffle grouping

Shuffle Grouping is a commonly employed stream grouping scheme for stateless operators. It assumes that the execution

Fig. 1 The proportion of the average aggregation time to the average processing time



time of all tuples is approximately equal, making it highly effective under this assumption. However, in practical streaming scenarios, data consistency is not always guaranteed. To address situations where the assumption does not hold, Rivetti et al. introduced Online Shuffle Grouping (OSG) (Rivetti et al. 2016). OSG utilizes the count-min sketch algorithm (Cormode and Muthukrishnan 2005) to estimate tuple execution time and a heuristic algorithm for tuple allocation.

For large-scale Location-Based Services (LBS) applications on the Apache Storm platform, Zhang et al. (2018) devised a novel stream partitioning strategy called Global Shuffle Grouping (GSG) to address continuous range query tasks. This strategy estimates the cost associated with different range queries and partitions query results based on this cost, aiming to achieve improved load balancing. In addition, the study proposed a cost estimation model for spatial queries and extended its application to other types of spatial queries.

2.2 Key grouping

Nasir et al. (2015a) introduced Partial Key Grouping (PKG) as a solution for handling skewed data streams within the context of stream processing. The main concept of PKG is to categorize key-value pairs into high-frequency and low-frequency segments. Low-frequency keys are processed using a conventional hash routing algorithm, while high-frequency keys are separated using the approach of The

Power of Two Choices (POTC) (Lumetta and Mitzenmacher 2007). Specifically, two distinct hash functions are employed to calculate two downstream nodes, and the node with the lighter load is selected for data transmission. Research studies have demonstrated that PKG effectively addresses load imbalance among downstream operators in the presence of data skewness. However, in cases where data skewness is substantial, utilizing only two downstream operators remains insufficient to achieve optimal load balancing. Consequently, Nasir et al. (2015b) proposed subsequent enhancement algorithms: W-choices and D-choices, to provide additional parallel instances as candidate choices for high-frequency keys.

Rivetti et al. (2015) introduced the Distribution-aware Key Grouping (DKG) method, which aims to perceive and learn the distribution characteristics of input data streams and achieve optimal load distribution for specific streams. The DKG method comprises three main stages: learning, construction, and deployment. In the learning stage, DKG acquires knowledge of the distribution characteristics of input data streams and employs the Space Saving algorithm (Metwally et al. 2005) to separate high-frequency keys from low-frequency keys. During the construction stage, high-frequency keys are eliminated, and statistics are computed for the remaining low-frequency keys. The routing table for high-frequency keys is constructed using the Shortest Processing Time First algorithm. In the final deployment stage, data tuples are distributed based on the routing table and hash functions. It should be noted that DKG is an offline

algorithm and lacks support for parallel processing, which restricts its practical use and limits its applicability to a theoretical approximation.

Balkesen et al. (2013) proposed a hash-based stream data partitioning method to achieve load balancing. In their study, tuples associated with hot keys are evenly distributed among multiple partitions, with the number of partitions proportional to the frequency of the keys. They employ sampling techniques to estimate the frequency of hot keys. However, the accuracy of the sampling-based scheme is suboptimal in distributed processing systems. Moreover, their design lacks adaptability to the dynamic changes in key frequencies observed in real-world streaming data. Based on the BiStream (Lin et al. 2015) framework, Zhang Zhang et al. (2019) introduced simois, a distributed connection system. Simois uses a set of important keys (called hotspots) identified from the workload to identify and optimize join queries, thereby reducing the imbalance caused by workload skewness. It should be noted that Zhang's method is specifically designed for handling join systems, limiting its applicability to other types of data processing tasks.

Chen et al. (2021) introduced a lightweight predictor called PStream for the identification of hot keywords in real-time data streams and efficient scheduling, which is similar to Aslam et al. (2021). The PStream system comprises two key components: an independent prediction component and a scheduling component integrated within each processing element instance. The prediction component employs a coin-flipping technique for conducting counting experiments, where the expected value increases for frequently occurring keys and decreases for infrequent ones. Through analysis of experimental outcomes, the hot keys within the data stream can be determined. Furthermore, the prediction results are stored in a Bloom filter, and the experimental values are probabilistically updated to accommodate changes in the data stream, thereby facilitating more precise dynamic hot key prediction. Leveraging the outcomes from the hot key prediction component, keys with lower frequencies are distributed using Key Grouping, while Shuffle Grouping is utilized for high-frequency keys. Zhang et al. (2023) proposed CompressStreamDB, a compression-based stream processing engine that enhances performance by enabling adaptive fine-grained processing on compressed data.

Data partitioning algorithms poses various challenges. A partitioner must have a low time complexity to meet the latency demands of streaming applications, and the algorithm must make quick judgments to prevent becoming a system performance bottleneck. Additionally, it is necessary to optimize the performance of downstream instances by implementing load balancing to obtain greater benefits from parallel downstream instances. Furthermore, a Key-Splitting partitioner should avoid scattering incoming tuples among too many downstream instances, because the aggregation

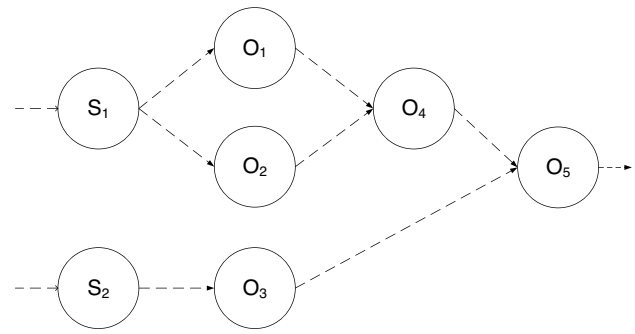


Fig. 2 An example of logical representation of a DAG

time could occupy most of the data processing time, resulting in system performance degradation.

To address these challenges, the FlexD partitioner is proposed. This partitioner classifies incoming tuples into low-frequency and high-frequency types and applies a hybrid processing technique as previous work (Gedik 2014; Pacaci and Özsu 2018; Fang et al. 2017). Tuples with low frequency are directly distributed by hash mapping, while tuples with frequency are constrained by adaptive Key-Splitting before distribution.

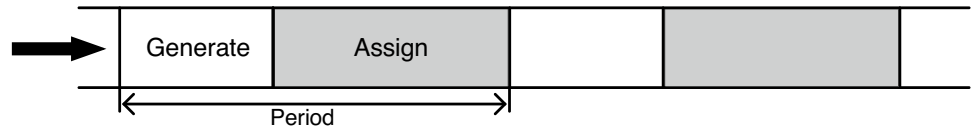
3 System model and problem definition

3.1 System model

In a classic stream processing model, a DAG is employed to describe the system (Nasir et al. 2015a), with nodes representing operators and edges representing the flow of data. Operators are parallelized into multiple instances based on the specified parallelism, as shown in Fig. 2. The data stream, denoted as S , consists of an infinite sequence of tuples. In the context of the sliding-window model, we use S^w to refer to the tuples within the logical window w , which can be defined based on counting or time criteria. Each tuple is represented as a triplet $t = (\tau, k, v)$, where τ denotes the tuple's order within the window w , and k and v represent the key and value associated with the tuple, respectively. A stream partition function $P : S^w \rightarrow \{W_1, W_2, \dots, W_n\}$, where $W_i \subset W$, specifies the downstream worker to which each tuple t in S^w is assigned based on its key k .

3.2 Problem definition

When the partitioning key of a stream is distributed unevenly, load balancing for stateful operator instances can present an intractable problem. Skewed data streams can lead to poor performance of a hash-based key grouping method, with some instances overloaded and others

Fig. 3 Timeline of operator

underutilized. The slowest instance that bears the heaviest workload can become a bottleneck in the system, hindering overall pipeline progress. Imbalances in memory, computation, and communication can also contribute to this bottleneck (Gedik 2014).

Definition 1 Let L_i^τ denote the load of each instance at time τ in terms of the number of tuples as follows:

$$L_i^\tau = |S_i^w|, i \in [1, |W|] \quad (1)$$

Definition 2 We define load imbalance λ based on (Pacaci and Özsu 2018) as follows:

$$\lambda^\tau = \frac{\max_{i \in [1, |W|]} (L_i^\tau) - \bar{L}}{\bar{L}} \quad (2)$$

where $\bar{L}$ is the mean load over all the instances.

For the Key-Splitting stream partitioning function P , it is likely that tuples with the same key k are assigned to non-empty downstream workers $W^k \subset W$ during the key splitting process. Each worker $W_i \in W^k$ upholds a partial state for the key k , resulting in the distribution of partial states across the system. We define replication factor γ as a metric to quantify the level of dispersion for each key k , representing the number of workers utilized to maintain the partial state for key k . This metric evaluates the degree of key separation and the resource cost required to maintain partial state and aggregate the final state.

Definition 3 We define replication factor γ as follows:

$$\gamma^\tau = \frac{\sum_k (|W^k|)}{K} \quad (3)$$

where K symbolize the cardinality of the set of unique keys in window S^w .

The partitioning function, denoted as $P = \langle Pt, Ph \rangle$, is constructed by combining the explicit routing table and the hash function, as proposed in previous works (Gedik 2014; Fang et al. 2017; Pacaci and Özsu 2018). Pt represents the explicit route table that maintains the mapping pairs of keys belonging to heavy hitters and the indexes of operator instances. The size of the route table is proportional to the number of heavy hitters. Ph is the hash function used for $|W|$ instances of the target operator. In this paper, we

utilize a 2-universal hash function (Rivetti et al. 2015) as an example.

4 FlexD key grouping

FlexD is a key grouping implementation that incorporates the micro-batch processing method. This method treats a potentially unbounded stream as a series of bounded segments and constructs a routing map for each segment. These mappings promote good load balance among operator instances within each segment. Based on the results, FlexD can provide a better load distribution for the entire stream.

4.1 Data stream model

In this study, we present our FlexD method within the context of the data stream model (Muthukrishnan 2003; Akidau et al. 2015). In this model, a stream consists of an infinite sequence of elements referred to as tuples or items, denoted as $\langle t_1, t_2, \dots, t_{xm} \rangle$. These items are extracted from a large universal set $[n] = \{1, 2, \dots, n\}$, where xm represents the unknown length or size of the stream. We use p_t to denote the unknown probability of item t appearing in the stream, and f_t represents its unknown frequency.

4.2 FlexD design

FlexD periodically processes each segment of the input stream, as shown in Fig. 3. Each data segment is considered as an operational period consisting of two phases: Generate and Assign. In the Generate phase, the operator sends the keys of collected tuples to the scheduler, and then the scheduler builds a new routing table according to Sketch and returns it to the operator. After receiving the routing table from the scheduler, the operator distributes the tuples to downstream instances according to the mapping defined in the routing table. After the distribution is completed, the system enters the next operational period. To reduce the impact of frequent communication between the operator and the scheduler on system performance, a method similar to micro-batching is used, where the operator collects at least m tuples before sending them to the Scheduler. This approach ensures that the communication frequency is not excessive while maintaining a high QoS level.

FlexD comprises two key components, operator and scheduler, whose workflows is modeled as finite state

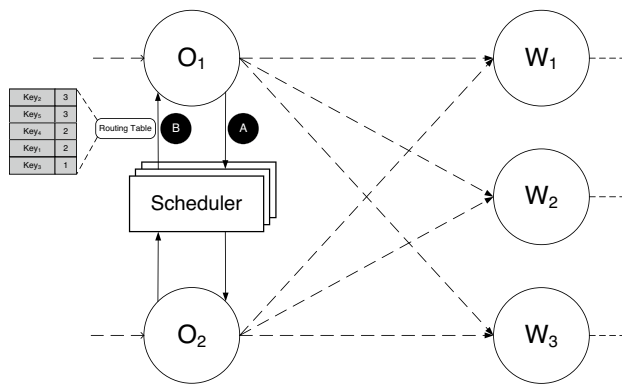


Fig. 4 Overview of FlexD design

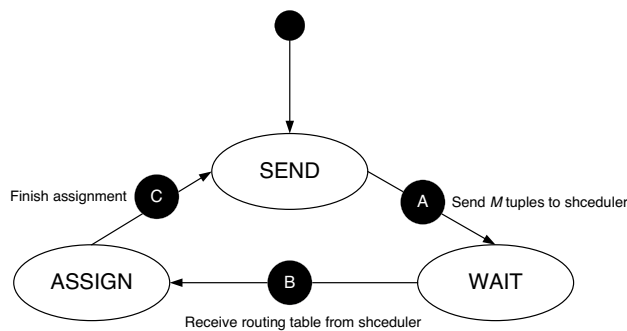


Fig. 5 Operator finite state machine

machines as shown in Fig. 4. The operator is characterized by three states, namely COLLECT, SEND, and ASSIGN, as shown in Fig. 5. Upon entering the COLLECT state, the operator collects information on m tuples and advances to the SEND state. In the SEND state, the operator awaits a routing table from the scheduler (as shown in Fig. 4A). Once the operator receives the routing table P_t (as shown in Fig. 4B), it proceeds to the ASSIGN state, distributing the tuples according to the mapping relations specified in the routing table, and returns to the COLLECT state.

The scheduler is similarly modeled as a finite state machine featuring three states, namely WAIT ALL, SEND, and RUN, as shown in Fig. 6. After entering the WAIT ALL state, the scheduler waits for information on the collected tuples from the operator and proceeds to the RUN state upon reception. In the RUN state, the scheduler generates a routing table based on the sketch information and FlexD algorithm, and moves to the SEND state. After returning the routing information to the corresponding operator, the scheduler returns to the WAIT ALL state. In this design, we can easily acquire global information of the system, enabling the FlexD algorithm to make better decisions. Meanwhile, to update the load status of downstream instances, multiple

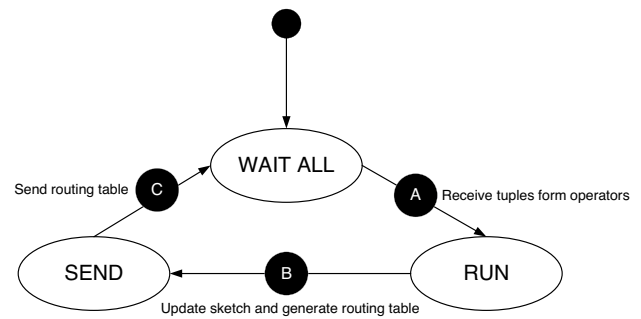


Fig. 6 Scheduler finite state machine

Schedulers synchronize information with each other over specific time intervals.

4.3 Sketch of stream

The FlexD algorithm systematically partitions an infinite data stream into substreams of a predetermined size l . To accurately infer the probability distribution within each substream segment, we incorporate the Space Saving algorithm (Metwally et al. 2005) to identify heavy hitters.

The Space Saving algorithm operates on the principle of deterministic counting to provide precise frequency estimations of items within the data stream. It is governed by the following parameters:

- *Stream segment size (l)*: This defines the size of each segmented portion of the data stream on which the algorithm operates. The algorithm considers each segment of size l independently to identify and estimate the frequency of heavy hitters within that segment.
- *Counter threshold (θ)*: Denoted by θ , this parameter establishes the threshold for the frequency counter. Items with a frequency above this threshold are considered significant and are tracked by the algorithm.
- *Error bound (ϵ)*: The parameter ϵ represents the permissible margin of error for frequency estimations. It ensures that the true frequency f_i of an item and its estimated frequency $\hat{f}_i$ satisfy the relationship $\hat{f}_i - f_i \leq \epsilon l$. The relationship between θ and ϵ is such that $0 < \epsilon < \theta \leq 1$.
- *Counter pair structure*: The algorithm maintains a set of $\lceil 1/\epsilon \rceil$ pairs, each pair being of the format $\langle \text{item}, \text{counter} \rangle$. This structure aids in tracking the frequencies of significant items and ensuring that the error bounds are adhered to.

Through rigorous mathematical validation (Rivetti et al. 2015), it is established that for any ϵ within its domain, the discrepancy between the actual and estimated frequencies of heavy hitters is bounded as previously described.

4.4 FlexD algorithm

Given that extending more downstream processing instances for high-frequency keys can result in better load balancing and performance, but also considering that excessive extension can lead to higher aggregation costs, FlexD proposes an algorithm based on the following idea: in the initial stage, it is easy to extend downstream processing instances for high-frequency keys, but as the number of processing instances increases, it becomes increasingly difficult to extend new downstream processing instances. Specifically, for an incoming tuple $t = (\tau, k, v)$, the FlexD algorithm uses a sketch (line 1) to query its frequency f_t within a given time window and then compares it with the high and low frequency threshold f (line 10). If $f_t < f$, it is considered a low-frequency tuple and directly mapped to a downstream operator instance through a hash function $\bar{h}$ (line 11). If $f_t \geq f$, it is considered a high-frequency tuple, and FlexD queries the frequency mapping table T_f to determine whether there is a value for

k (line 13). If not, k is considered to be a high-frequency key for the first time and a downstream instance W_i with the lightest load is allocated for it (lines 14–16). If there is an entry k in T_f , FlexD checks whether f_t is greater than $T_f[k]$. If $f_t < T_f[k]$, it queries the instance mapping table and selects the downstream instance with the lightest load (line 22). If $f_t \geq T_f[k]$, it means that key k has exceeded the frequency threshold again, and FlexD adds another unused downstream instance W_j which has the lightest load of k , and updates $T_f[k] = T_f[k] * \delta$ (lines 18–20). The pseudocode of the FlexD algorithm is presented in Algorithm 1.

Additionally, because we used two dictionaries in the algorithm (lines 4–5) to record the frequency threshold and mapping instance corresponding to key k , these two dictionaries inevitably grew larger over time. To control memory usage, we implemented a mechanism for regularly clearing entries corresponding to keys that had not been accessed for a long time. Specifically, we cleared and retained the top 50k records every 10 min.

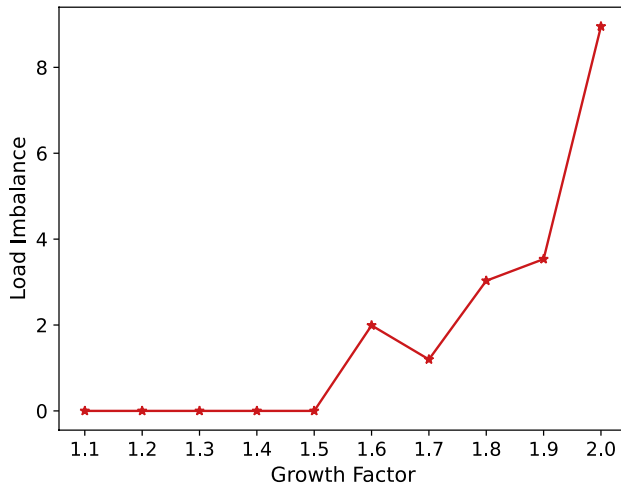
Algorithm 1 Scheduler

Input: Incoming tuples *Tuples*
Output: Routing table for tuples **RT**

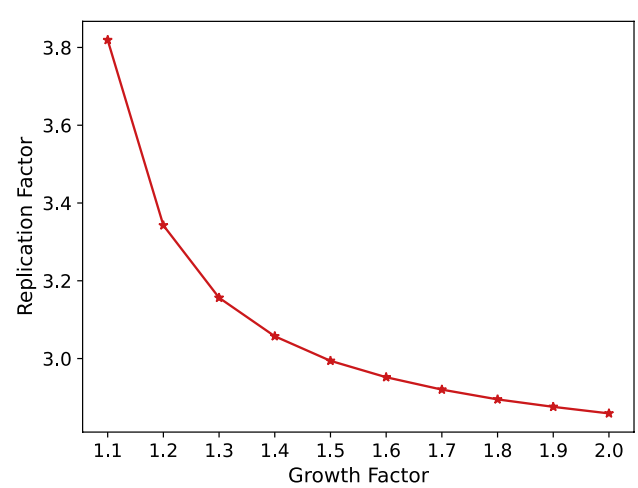
- 1: **SS**: SpaceSaving instance with parameter θ, ϵ
- 2: **HB**: Buckets for recording historical load of each downstream instances
- 3: $\bar{h}$: A 2-universal hash function
- 4: T_f : Dictionary for recording the threshold frequency of high-frequency keys
- 5: T_v : Dictionary for recording mapped instances of high-frequency keys
- 6: f : Frequency threshold
- 7: **SS.update**(*Tuples*)
- 8: **for all** *tuple* $\in$ *Tuples* **do**
- 9: $f_t = \text{SS.query}(\text{tuple})$
- 10: **if** $f_t < f$ **then**
- 11: $v = \bar{h}(\text{tuple})$
- 12: **else**
- 13: **if** T_f not contains *tuple* **then**
- 14: $v = \text{findLeastLoad}(\text{HB})$
- 15: $T_f[\text{tuple}] = f$
- 16: $T_v[\text{tuple}] += v$
- 17: **else if** $f_t \geq T_f[\text{tuple}]$ **then**
- 18: $v = \text{extendInstance}(\text{HB}, T_v)$
- 19: $T_f[\text{tuple}] = T_f[\text{tuple}] * \delta$
- 20: $T_v[\text{tuple}] += v$
- 21: **else**
- 22: $v = \text{findLeastLoad}(T_v)$
- 23: **end if**
- 24: **end if**
- 25: $\text{RT}[\text{tuple}] = v$
- 26: **end for**

Table 1 Parameters used in our experiments

Parameter	Description	Default Value
θ	Size of each segment of data stream	0.01
ϵ	Permissible margin of error for frequency estimations	$\theta/2$
δ	Growth factor for scaling the frequency threshold	1.5
m	Tuples sent each round	1000
z	Coefficients of zipfian distributions	[1.0, 3.0]



(a) Load Imbalance



(b) Replication Factor

Fig. 7 Impact of growth factor δ on the performance of FlexD

5 Analysis

Tuple collection complexity. For each operator O in FlexD during a given period, the time complexity $T_{\text{collection}}$ required for tuple collection is defined as:

$$T_{\text{collection}}(O) = O(m)$$

It should be noted that the actual time can vary based on the prevailing network rate.

Tuple assignment complexity. Upon the receipt of the routing table, the time complexity $T_{\text{assignment}}$ for an operator to allocate the tuples is:

$$T_{\text{assignment}}(O) = O(m)$$

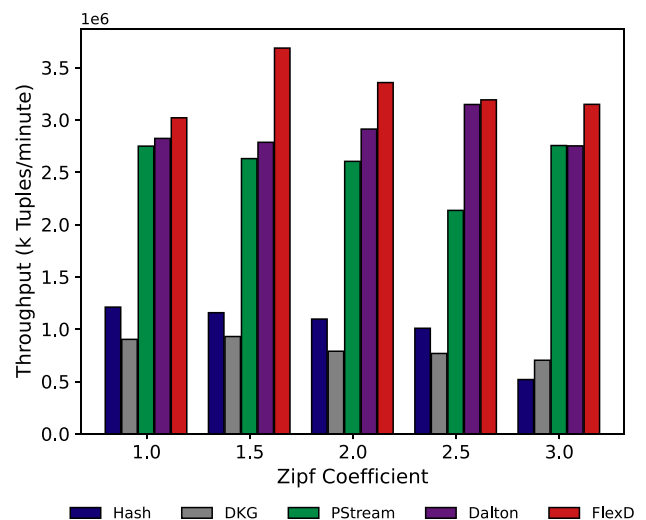
Routing table construction complexity. For each scheduler S in FlexD, the time complexity T_{routing} for creating a new routing table is defined as:

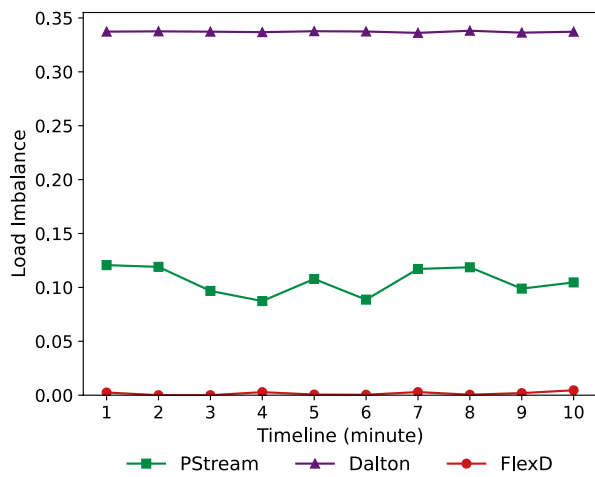
$$T_{\text{routing}}(S) = O(m \times (\lfloor 1/\epsilon \rfloor + |W|))$$

Memory overhead of spacesaving. The algorithm *SpaceSaving* requires a space complexity of $O(\lfloor 1/\epsilon \rfloor)$ for workload estimation.

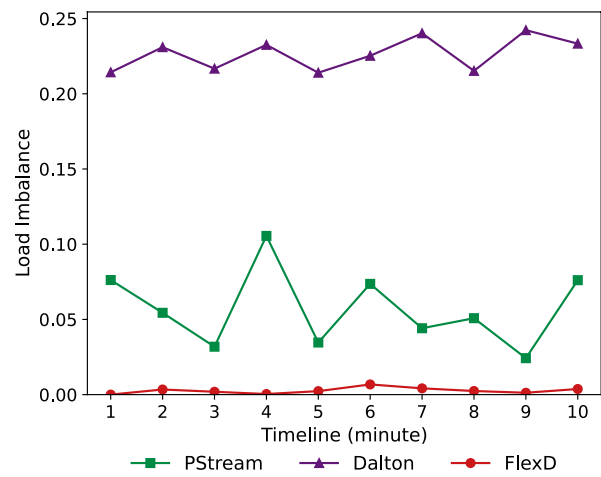
Memory complexity for tuple storage in FlexD. For each scheduler S in a given period, the memory complexity M_{store} required by FlexD to store tuples is defined as:

$$M_{\text{store}}(S) = O(m)$$

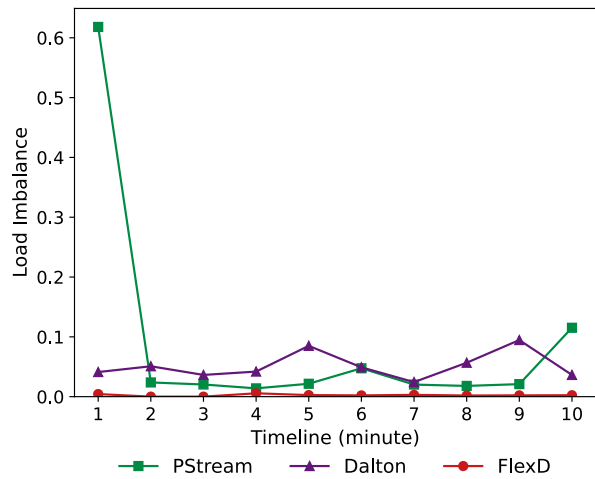
**Fig. 8** Throughput ϕ in different levels of skewness



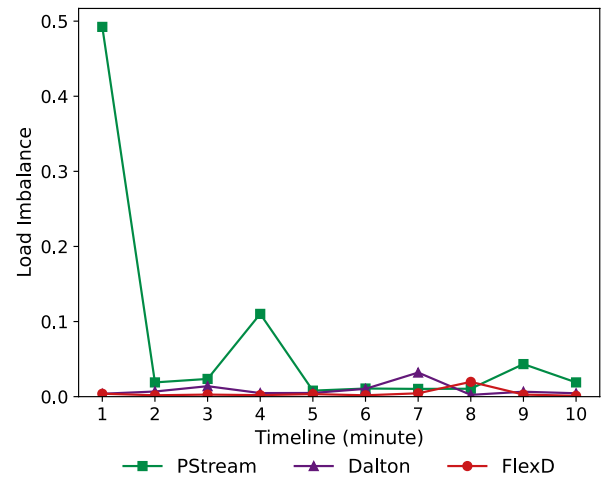
(a) Zipf coefficient = 1.0



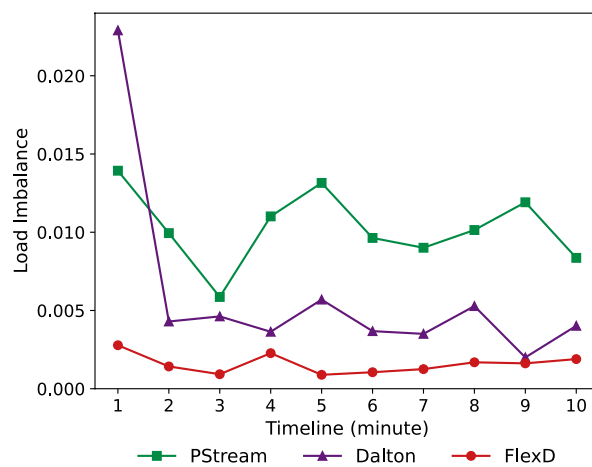
(b) Zipf coefficient = 1.5



(c) Zipf coefficient = 2.0

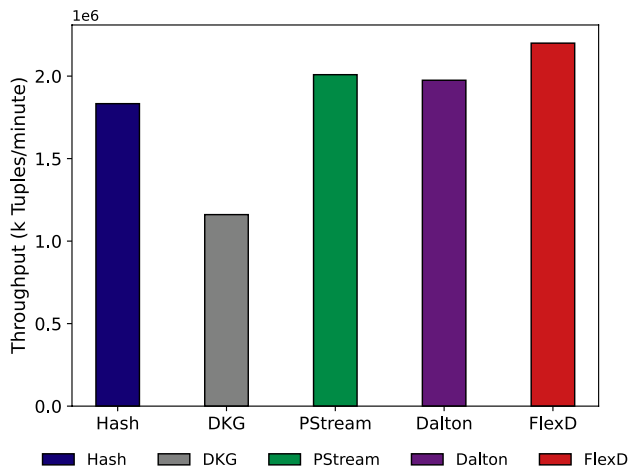


(d) Zipf coefficient = 2.5

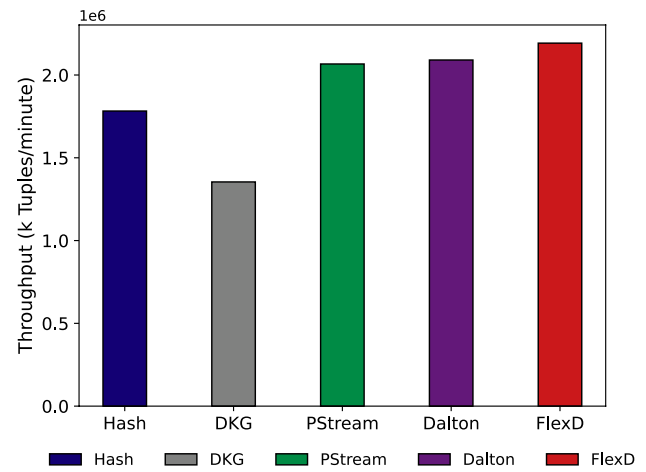


(e) Zipf coefficient = 3.0

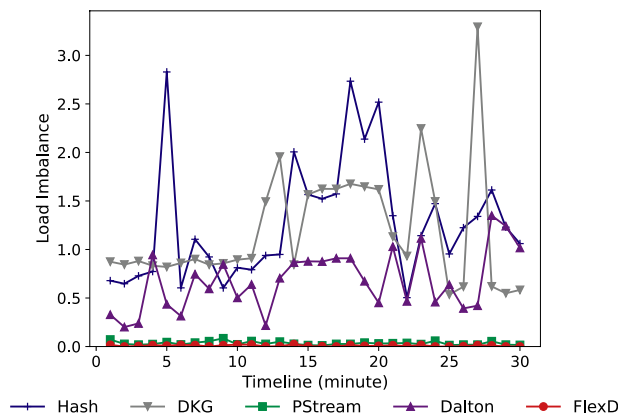
Fig. 9 Load imbalance λ in different levels of skewness



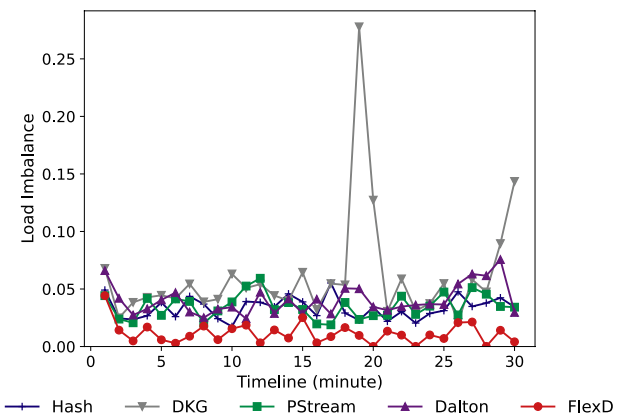
(a) Voters dataset



(b) Amazon reviews dataset

Fig. 10 Throughput ϕ on T4SA and Wikitext dataset

(a) Voters dataset



(b) Amazon reviews dataset

Fig. 11 Load imbalance as a function on Voters and Amazon dataset

Memory complexity for high-frequency key mapping in FlexD.: The memory complexity M_{map} required to record the mapped instances of high-frequency keys is:

$$M_{map}(S) = O(m \times |W|)$$

Memory complexity for frequency threshold in FlexD.: The memory complexity $M_{threshold}$ required to record the frequency threshold of high-frequency keys is:

$$M_{threshold}(S) = O(m)$$

6 Evaluation

In this section, we evaluate the performance obtainable by using FlexD for key grouping and compared it with other key grouping solutions.

The parameter $\delta \in (1, \infty]$ is used to adjust the splitting speed in FlexD. The interplay between load imbalance and the replication factor in FlexD, as influenced by δ , is shown in Fig. 7. Notably, a diminution in δ enhances load balance but incurs a higher rate of splitting. This trend is

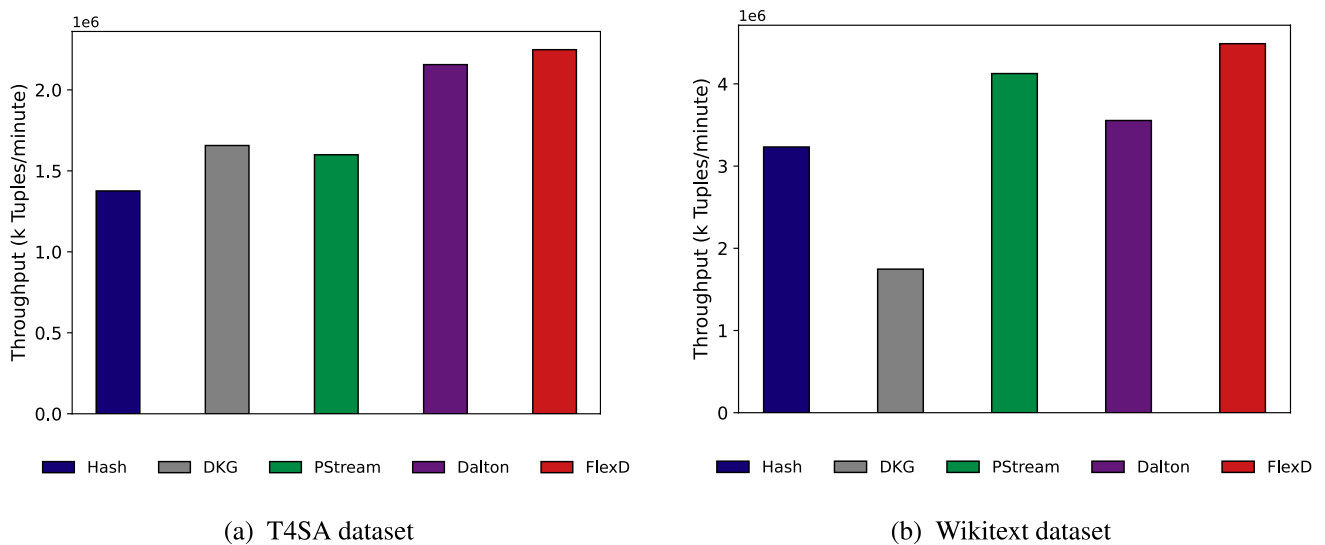


Fig. 12 Throughput ϕ on T4SA and Wikitext dataset

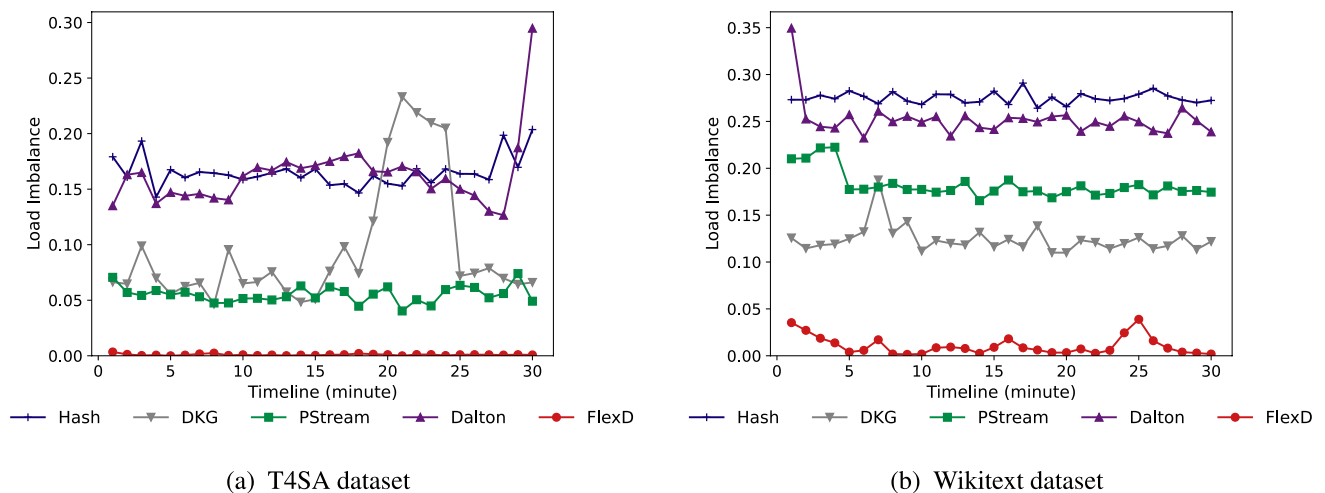


Fig. 13 Load imbalance as a function on T4SA and Wikitext dataset

expected: as δ increases, FlexD places stronger restrictions on partitioning to avoid excessive key-value splits. Based on these observations, we adopt a default value of $\delta = 1.5$, which best coordinates the two competitive metrics.

6.1 Experimental setup

6.1.1 Datasets

We have tested both synthetic and real datasets in our experiments. For the synthetic datasets, we generated

streams of integer values representing the values of the tuples. The synthetic dataset has been generated following Zipfian distributions with different coefficients z varying from 1.0 to 3.0 (Table 1).

For the real dataset, we utilized the Amazon Customer Reviews Dataset,¹ which consists of millions of customer reviews for products on the Amazon.com website. The Amazon Customer Reviews Dataset includes over 1.3 million user reviews, presenting evaluations and expressions of customers' product experiences, including the star rating. To analyze consumers' perspectives and preferences towards products, we devised an experiment to identify and collect information about groups of products with high and low

¹ <https://s3.amazonaws.com/amazon-reviews-pds/readme.html>

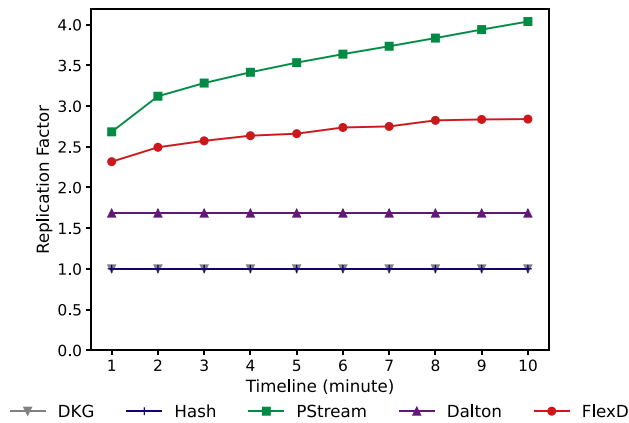


Fig. 14 Replication factor γ on Synthetic dataset (zipf ~3.0)

ratings. Each comment is represented as a tuple, with the product ID serving as the key and all associated data for the product as the value. The second is the voters dataset,² which represents the voter registration data for North Carolina. We use the postcode as the key and aggregate registration data from different voters in various regions. The remaining two datasets are T4SA (Vadicamo et al. 2017) and WikiText (Merity et al. 2016). T4SA consists of approximately 3.4 million tweets, while WikiText is a collection of over 100 million tokens extracted from Good and Featured articles in Wikipedia. We use these two datasets for the word count application.

6.1.2 Compared methods

We compare the performance of FlexD with:

1. *1-choice partitioners* distribute all tuples to the same worker corresponding to their keys, eliminating any aggregation time overhead. For such group, we choose Hash³ and DKG (Rivetti et al. 2015) as the partitioning functions.
2. *N-choice partitioners* split keys to different downstream workers based on a specific strategy, resulting in varying aggregation time overhead depending on the degree of Key-Splitting. For such group, we choose PStream Chen et al. (2021) and Dalton Zapidou et al. (2022) as the partitioning functions.

6.1.3 Evaluation metrics

The evaluation metrics are

² <https://www.ncsbe.gov/results-data/voter-registration-data>

³ We implement Hash solution using the Murmurhash function provided by Guava.

- *Load imbalance* (λ) measures the performance loss in the assignment phase due to imbalanced load assignment with respect to perfectly balanced assignment. It is defined in Eq. (2).
- *Replication factor* (γ) measures the degree of key splitting and the resource cost required for performing aggregation during the aggregation phase. It is defined in Eq. (3).
- *Throughput* (ϕ) measures the effectiveness of stream partitioning solutions in tuples processed per minute.

6.1.4 Experiments

The FlexD algorithm was implemented as a custom stream grouping on Apache Storm (<http://storm.apache.org/>). Experiments on comparisons of the solutions were conducted on an 8-node cluster, with each node consisting of 12 cores and 8 GB of RAM, and each worker node having a single available slot.

6.2 Results

Figure 8 shows the throughput of the system under different levels of data skewness. We compare FlexD with other solutions, maintaining number of upstream and downstream operators as well as tuple sending rates, while adjusting the skewness of the input data. The results indicate that FlexD outperforms the other schemes with a higher throughput and more stable output performance. It is noteworthy that DKG has the lowest throughput performance since it does not support parallel processing. In addition, most solutions experience a decrease in throughput with an increase in skew coefficient, which is consistent with the basic characteristics of (Chen et al. 2021). Data skewness may lead to imbalanced system load, which may lead to performance degradation. Compared to other baselines, FlexD has better load balancing ability, and therefore exhibits a more stable performance decline.

Figure 9 show the substantial advantage of FlexD over other baseline approaches in addressing data skewness. To ensure the accuracy of comparisons with other algorithms, we will omit the load imbalance comparison of Hash and DKG, as they exhibit significant load imbalance. PStream and Dalton demonstrate good load balancing performance at lower levels of data skewness, and the overall trend is relatively stable. However, as the data skewness increases, PStream exhibits higher load imbalance in the initial stage due to the delayed detection of high-frequency keys by its hot key predictor, particularly when the Zipf coefficient is 2.0 and 2.5. When the Zipf coefficient is 3.0, both PStream and Dalton experience the aforementioned phenomenon simultaneously. In contrast, FlexD exhibits good load

balancing performance across different Zipf coefficients, maintaining a stable overall trend.

Figures 10 and 11 show the throughput and load balancing performance of different schemes on the Voters and Amazon datasets, respectively. The results indicate that FlexD outperforms other solutions slightly in complex tasks and exhibits the lowest load imbalance among all schemes. It is noteworthy that, on the Voters dataset, although FlexD and PStream display almost identical load balancing performance, PStream incurs higher aggregation cost due to leveraging Shuffle Grouping for high-frequency keys, resulting in slightly lower throughput than FlexD. On the Amazon dataset, a similar phenomenon is observed: PStream and Dalton exhibit similar load imbalance. However, Dalton has a lower aggregation cost, resulting in slightly higher throughput than PStream. On the other hand, FlexD strikes a good balance between load imbalance and aggregation cost and hence outperforms the former two solutions in terms of processing throughput.

Figures 12 and 13 present the throughput and load balancing performance on the T4SA and Wikitext datasets. The results indicate that, compared to other approaches, FlexD performs similarly or slightly better in the simple Word-Count task. The T4SA data has a uniform distribution, while the Wikitext data presents a higher degree of skewness. The 1-choice partitioners hash mapping scheme performs well on uniformly distributed data, but cannot expand processing nodes like the N-choice partitioners in skewed data, resulting in lower performance. Notably, observations during the experiment indicate that Dalton often allocates more keys to a specific node to achieve higher reward scores, leading to poorer load balancing performance on the Wikitext dataset. Overall, FlexD achieves high load balancing and throughput in simple tasks.

In addition, we evaluated the replication factors of different solutions on a synthetic dataset. Figure 14 presents the replication factors of different schemes in the synthetic dataset with a bias coefficient of 3.0. For the 1-choice partitioners, its replication factor remains 1. However, for the N-choice partitioners, as this scheme splits keys and distributes the load to downstream operators, the replication factor can also be used to evaluate the aggregation cost of the scheme to a certain extent. The results show that although Dalton splits keys, it cannot dynamically adjust the degree of splitting to adapt to the change of data streams. Its replication factor is usually linear, thus cannot effectively deal with the change of data streams, which also affects the load balancing of the system. In contrast, both PStream and FlexD can dynamically adjust according to changes in data streams. However, the PStream scheme is more aggressive and can lead to higher aggregation costs. Although FlexD's replication factor does not significantly increase as PStream does,

it has the ability to dynamically adjust and maintains good load balancing performance.

It is noteworthy that FlexD's ability to handle load imbalance and deviation is crucial in real-world scenarios, making it a viable option for applications that require handling large-scale datasets while maintaining performance stability.

7 Conclusion

Key-Splitting is an efficient solution for enhancing parallel workloads in stream processing systems. However, excessive key separation may lead additional overhead in aggregation, resulting in a reduction in the overall system throughput. To address this issue, we present an adaptive Key-Splitting algorithm, FlexD, that is capable of adapting to fluctuations in the data stream. This algorithm aims to alleviate load imbalance by partitioning keys among downstream operators, gradually increasing the degree of key separation while also introducing higher difficulty in splitting to limit excessive partitioning. We have implemented this algorithm on the Apache Storm platform and conducted experimental evaluations. The experimental results demonstrate that our algorithm outperforms existing designs in terms of load balancing and throughput.

Acknowledgements This work is partially supported by the following grants: the 14th "115" Industrial Innovation Group (Project 4) of Anhui Province, NSFC 62072311, U2001212, Guangdong Project 2020B1515120028, Shenzhen Project JCYJ20210324094402008. Amelie Chi Zhou's work is partly supported by NSFC 62172282, Shenzhen STI Project RCYX20221008092908029 and a startup grant of HKBU.

Funding Open access funding provided by Hong Kong Baptist University Library.

Declarations

Conflict of interest On behalf of all authors, the corresponding author states that there is no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Abdelhamid, A.S., Aref, W.G.: Partly: learning data partitioning for distributed data stream processing. In: Proceedings of the Third International Workshop on Exploiting Artificial Intelligence Techniques for Data Management, pp. 1–4 (2020)
- Abdelhamid, A.S., Mahmood, A.R., Daghistani, A., Aref, W.G.: Prompt: Dynamic data-partitioning for distributed micro-batch stream processing systems. In: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, pp. 2455–2469 (2020)
- Akidau, T., Bradshaw, R., Chambers, C., Chernyak, S., Fernández-Moctezuma, R.J., Lax, R., McVeety, S., Mills, D., Perry, F., Schmidt, E., Whittle, S.: The dataflow model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proc. VLDB Endow.* **8**(12), 1792–1803 (2015)
- Apache Storm. <http://storm.apache.org/>. Accessed Jan 2024
- Aslam, A., Chen, H., Jin, H.: Pre-filtering based summarization for data partitioning in distributed stream processing. *Concurr. Comput. Pract. Exp.* **33**(20), 6338 (2021)
- Balkesen, C., Tatbul, N., Özsu, M.T.: Adaptive input admission and management for parallel stream processing. In: Proceedings of the 7th ACM International Conference on Distributed Event-based Systems. DEBS '13, pp. 15–26. ACM, New York, NY, USA (2013)
- Chen, H., Zhang, F., Jin, H.: Pstream: a popularity-aware differentiated distributed stream processing system. *IEEE Trans. Comput.* **70**(10), 1582–1597 (2021)
- Cormode, G., Muthukrishnan, S.: An improved data stream summary: the count-min sketch and its applications. *J. Algorithms* **55**(1), 58–75 (2005)
- Fang, J., Zhang, R., Fu, T.Z.J., Zhang, Z., Zhou, A., Zhu, J.: Parallel stream processing against workload skewness and variance. In: Proceedings of the 26th International Symposium on High-Performance Parallel and Distributed Computing. HPDC '17, pp. 15–26. ACM, New York, NY, USA (2017)
- Gedik, B.: Partitioning functions for stateful data parallelism in stream processing. *VLDB J.* **23**(4), 517–539 (2014)
- Gedik, B., Schneider, S., Hirzel, M., Wu, K.: Elastic scaling for data stream processing. *IEEE Trans. Parallel Distrib. Syst.* **25**(6), 1447–1463 (2014)
- Katsipoulakis, N.R., Labrinidis, A., Chrysanthos, P.K.: A holistic view of stream partitioning costs. *Proc. VLDB Endow.* **10**(11), 1286–1297 (2017)
- Lin, Q., Ooi, B.C., Wang, Z., Yu, C.: Scalable distributed stream join processing. In: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data. SIGMOD '15, pp. 811–825. Association for Computing Machinery, New York, NY, USA (2015)
- Lumetta, S., Mitzenmacher, M.: Using the power of two choices to improve bloom filters. *Internet Math.* **4**(1), 17–33 (2007)
- Merity, S., Xiong, C., Bradbury, J., Socher, R.: Pointer sentinel mixture models. <http://arxiv.org/abs/1609.07843> (2016)
- Metwally, A., Agrawal, D., El Abbadi, A.: Efficient computation of frequent and top-k elements in data streams. *ICDT'05*, pp. 398–412. Springer, Berlin, Heidelberg (2005)
- Muthukrishnan, S.: Data streams: Algorithms and applications. In: Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms. SODA '03, p. 413. Society for Industrial and Applied Mathematics, USA (2003)
- Nasir, M.A.U., Morales, G.D.F., Garcia-Soriano, D., Kourtellis, N., Serafini, M.: The power of both choices: practical load balancing for distributed stream processing engines. In: 2015 IEEE 31st International Conference on Data Engineering. IEEE (2015a)
- Nasir, M.A.U., Morales, G.D.F., Kourtellis, N., Serafini, M.: When two choices are not enough: balancing at scale in distributed stream processing. *CoRR arXiv:1510.05714* (2015b)
- Pacaci, A., Özsu, M.T.: Distribution-aware stream partitioning for distributed stream processing systems. In: Proceedings of the 5th ACM SIGMOD Workshop on Algorithms and Systems for MapReduce and Beyond. BeyondMR'18, pp. 6–1610. ACM, New York, NY, USA (2018)
- Qureshi, M.M., Chen, H., Jin, H.: Modeling distributed stream processing systems under heavy workload. In: 2019 International Conference on Cyberworlds (CW), pp. 93–100 (2019)
- Rivetti, N., Anceaume, E., Busnel, Y., Querzoni, L., Sericola, B.: Online scheduling for shuffle grouping in distributed stream processing systems. In: Proceedings of the 17th International Middleware Conference. Middleware '16, pp. 11–11112. ACM, New York, NY, USA (2016)
- Rivetti, N., Querzoni, L., Anceaume, E., Busnel, Y., Sericola, B.: Efficient key grouping for near-optimal load balancing in stream processing systems. In: Proceedings of the 9th ACM International Conference on Distributed Event-Based Systems. DEBS '15, pp. 80–91. ACM, New York, NY, USA (2015)
- Schneider, S., Hirzel, M., Gedik, B., Wu, K.: Auto-parallelizing stateful distributed streaming applications. In: 2012 21st International Conference on Parallel Architectures and Compilation Techniques (PACT), pp. 53–64 (2012)
- Toliopoulos, T., Gounaris, A.: Adaptive distributed partitioning in apache flink. In: 2020 IEEE 36th International Conference on Data Engineering Workshops (ICDEW), pp. 127–132 (2020). IEEE
- Vadicamo, L., Carrara, F., Cimino, A., Cresci, S., Dell'Orletta, F., Falchi, F., Tesconi, M.: Cross-media learning for image sentiment analysis in the wild. In: 2017 IEEE International Conference on Computer Vision Workshops (ICCVW), pp. 308–317 (2017)
- Xiao, Y., Chen, S., Zhou, A.C., Zhang, S., Wang, Y., Mao, R., Yang, X.: Low-Latency Video Conferencing System for Geo-Distributed Data Centers (2023)
- Zapridou, E., Mytilinis, I., Ailamaki, A.: Dalton: learned partitioning for distributed data streams. *Proc. VLDB Endow.* **16**(3), 491–504 (2022)
- Zhang, S., He, B., Dahlmeier, D., Zhou, A.C., Heinze, T.: Revisiting the design of data stream processing systems on multi-core processors. In: 2017 IEEE 33rd International Conference on Data Engineering (ICDE), pp. 659–670. IEEE (2017)
- Zhang, Y., Wang, B., Zhou, J., Zhong, H., Tian, X.: Global shuffle grouping (gsg): A load balancing strategy for continuous range queries on storm. In: 2018 IEEE 16th International Conference on Software Engineering Research, Management and Applications (SERA), pp. 71–78 (2018)
- Zhang, F., Chen, H., Jin, H.: Simois: a scalable distributed stream join system with skewed workloads. In: 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS), pp. 176–185 (2019)
- Zhang, S., He, J., Zhou, A.C., He, B.: Briskstream: scaling data stream processing on shared-memory multicore architectures. In: Proceedings of the 2019 International Conference on Management of Data, pp. 705–722 (2019)
- Zhang, Y., Zhang, F., Li, H., Zhang, S., Du, X.: Compressstreamdb: fine-grained adaptive stream processing without decompression. In: 2023 IEEE 39th International Conference on Data Engineering (ICDE), pp. 408–422. IEEE (2023)



Gang Liu is a lecturer at High Performance Computing Institute, College of Computer Science and Software Engineering, Shenzhen University, China. Before joining Shenzhen University, he was an engineer in ASML Brion Technologies Inc. He received his PhD and BS degree in computer science from the University of Science and Technology of China (USTC) in 2006 and 2001, respectively. His research interests lie in high performance computing, federated learning and bigdata

cloud computing, high performance computing, big data processing and resource management.

processing.

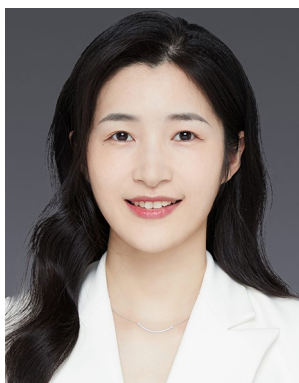


Zeting Wang is currently working toward the master's degree in ShenZhen University, advised by G. Liu. His research interests include network big data and data mining.



Rui Mao received BS (1997) and MS (2000) in computer science from the University of Science and Technology of China, and MS(2006) in statistics and PhD (2007) in computer science from the University of Texas at Austin. After 3 years work at the Oracle USA Corporation, he joined Shenzhen University in 2010. He is now a Distinguished Professor at the College of Computer Science and Software Engineering, Shenzhen University, and Executive Director of the Shenzhen Institute of Com-

puting Sciences. His research mainly focuses on metric-space data processing.



Amelie Chi Zhou is currently an Assistant Professor in Hong Kong Baptist University (HKBU), China. Before joining HKBU, she was an Associate Professor in Shenzhen University and a Postdoc Fellow in Inria-Bretagne research center, France. She received her PhD degree in 2016 from School of Computer Engineering, Nanyang Technological University, Singapore. Her research interests lie in